

1 Motivation

In this document, I will be outlining the scope of the computable semantic model by using a real example to motivate its features. The objective is to specify a real design, then reason about it and then use that to understand the scope of what the engine must be able to do. By doing this, the structure that needs to be implemented will reveal itself and there will be some underlying principles which guide it.

I will define basic computable semantic primitives and build the computable semantic model using that. The invariants specified by the CSP's will be enforced by reasoning about the design. For example, by going through this process, it will become clear that the validation mechanism will need to be able to find paths between nodes. In the same way, as I reason about the design I establish, I will make a list of features that I have to support.

This structured way of building this library will be more effective because it will force me to refine my thoughts and establish a structure that I will implement. It will also unambiguously define the scope of what is currently supported and how it can be expanded to support more features.

2 Design Abstraction Language

Currently, an AST library for the Design Abstraction Language (DAL) has been implemented with reserved keywords and structures, as well as the ability to specify custom commands. I am listing the features of the language in a general sense and then I will leverage them to meet the needs of the CSM.

2.1 Structure

The structural specification enables the the construction of a block using braces. For example, the block can be used as follows to specify control flow logic for a behavior.

```
1 select {
2     # Specify control flow logic
3 }
```

2.2 Commands or Attributes

The command structure itself is specified as shown below. Each command is specified and in the bracket the arguments are provided. There can be keyword commands used by the design and then there are custom commands that reference more meaning.

```
1 # Built In
2 design("Design Name")
3
4 # Custom Command
5 command(
6     number=123,
7     string="String",
8     arrayArg=[12,33,12],
9     objArg={"a":1}
10 )
11
12 # Reusing structure to specify attributes
13 property(
14     name="type"
15     description="Data type of participant (list, object, str etc)"
16     type="string"
17     get="getType"
18 )
```

2.3 Control Flow

The control flow structure itself is specified as specified below. Currently, only if statements are supposed but not else if or else blocks. I need to extend the library to support this.

```
1  # Currently supported
2  if (boolean) {
3
4  }
```

This basic structure makes it possible to specify all the necessary attributes without making the AST implementation itself complicated. As I work through the reasoning and build the CSM, I will be using these structures specify the necessary meaning. If I run into features I can't support, I will return here and add to it.

2.4 Abstract Syntax Tree

The DAL script is parsed into an Abstract Syntax Tree(AST) that is then used by the validator. An example of a simple AST generated by a DAL script is provided below. In a full design specification, multiple dal files can be used to specify the behavior attributes, participant attributes etc.

2.4.1 Sample DAL Script

```
1  design("test")
2
3  behavior getName {
4      select {
5          goToBehavior(addBook)
6      }
7  }
8
9  behavior addBook {
10     #...
11     select {
12         goToBehavior(getName)
13     }
14 }
```

2.4.2 Sample AST

```
1  {
2      "type": "design",
3      "body": [
4          {
5              "type": "behavior",
6              "behaviorName": "getName",
7              "body": [
8                  {
9                      "type": "select",
10                     "args": [],
11                     "body": [
12                         {
13                             "type": "cmd",
14                             "command": "goToBehavior",
15                             "args": [
16
```

```

17         "arg": null,
18         "type": "name",
19         "value": "addBook"
20     }
21 ]
22 }
23 ]
24 }
25 ]
26 },
27 {
28     "type": "behavior",
29     "behaviorName": "addBook",
30     "body": [
31         {
32             "type": "select",
33             "args": [],
34             "body": [
35                 {
36                     "type": "cmd",
37                     "command": "goToBehavior",
38                     "args": [
39                         {
40                             "arg": null,
41                             "type": "name",
42                             "value": "getName"
43                         }
44                     ]
45                 }
46             ]
47         }
48     ]
49 },
50 ],
51 "name": [
52     {
53         "arg": null,
54         "type": "string",
55         "value": "test"
56     }
57 ]
58 }

```

2.4.3 Modifying AST by injecting nodes

TODO — Implement feature to modify AST by injecting nodes

This is necessary because through the reasoning, the engine will make automatic modifications to the design by enforcing the designs own meaning. I am not talking about the actual types of changes yet, just the ability to add nodes.

3 Design - Library Manager

For the first design, I will be using a simple library manager because it exposes a lot of features while keeping the implementation simple. I will first informally discuss the meaning of the world and then I will formalize it using the design abstraction language. Then I will discuss the kinds of reasoning that can be done and identify the necessary features that the tool must have. Then I will describe how the meaning can be established using a semantic toolkit and automatically constructed as a closed semantic world.

4 Design - Bubble Sort

The same process will be repeated for a bubble sort algorithm to demonstrate this process using a familiar design. It is not a particularly difficult design but that is what makes it interesting because it gives an opportunity to practice applying the principles and establishing its meaning.

5 Design - Distributed Rendering

The same process will be repeated for a distributed design, in this case, it is a distributed rendering application.

In this design, there is:

- One node:
 - that interfaces with the client and accepts jobs on one process
 - assigns the jobs to the rendering machines on another process and also monitors that the jobs are completed
- One database that stores the jobs and their states
- Four rendering machines

The rendering jobs themselves are 3d files submitted by the user and they are split into X mb chunks and rendered one chunk at a time until its done and then it is merged back together and shipped.

This will be a very neat design because it will force me to really confront some more corner cases in this framework while also exploring a distributed design. The only design that I've explored in this process that has concurrency in it is the multi threaded text translator, so this will be interesting.